4-bit Array Multiplier: RTL to GDSII on SkyWater SKY130

Digital VLSI Assignment Report (Parts A–D)

Roll Numbers: 2025122003 and 2025122007

October 26, 2026

GitHub repository:

`https://github.com/DevangBordoloi/RTL-to-GDS-4-bit-array-multiplier-/tree/main`

1 Part A

1.1 Pass Summary Line

```
VCD info: dumpfile mult.vcd opened for output.
PASS: 256 vectors checked, 0 errors
```

1.2 Fail Summary Line

After changing `sum` in `full_adder.v` to `a ^ b`:

```
MISMATCH: a=3 b=3 expected=9 got=5 (time=1580000)
MISMATCH: a=3 b=6 expected=18 got=10 (time=1670000)
MISMATCH: a=3 b=7 expected=21 got=9 (time=1700000)
MISMATCH: a=3 b=11 expected=33 got=29 (time=1820000)
MISMATCH: a=3 b=12 expected=36 got=20 (time=1850000)
MISMATCH: a=3 b=13 expected=39 got=23 (time=1880000)
MISMATCH: a=3 b=14 expected=42 got=18 (time=1910000)
MISMATCH: a=3 b=15 expected=45 got=17 (time=1940000)
MISMATCH: a=5 b=5 expected=25 got=17 (time=2600000)
MISMATCH: a=5 b=7 expected=35 got=27 (time=2660000)
MISMATCH: a=5 b=10 expected=50 got=34 (time=2750000)
MISMATCH: a=5 b=11 expected=55 got=39 (time=2780000)
MISMATCH: a=5 b=13 expected=65 got=57 (time=2840000)
MISMATCH: a=5 b=14 expected=70 got=54 (time=2870000)
MISMATCH: a=5 b=15 expected=75 got=51 (time=2900000)
MISMATCH: a=6 b=3 expected=18 got=10 (time=3020000)
MISMATCH: a=6 b=6 expected=36 got=20 (time=3110000)
MISMATCH: a=6 b=7 expected=42 got=18 (time=3140000)
MISMATCH: a=6 b=11 expected=66 got=58 (time=3260000)
MISMATCH: a=6 b=12 expected=72 got=40 (time=3290000)
MISMATCH: a=6 b=13 expected=78 got=46 (time=3320000)
MISMATCH: a=6 b=14 expected=84 got=36 (time=3350000)
MISMATCH: a=6 b=15 expected=90 got=34 (time=3380000)
MISMATCH: a=7 b=3 expected=21 got=9 (time=3500000)
MISMATCH: a=7 b=5 expected=35 got=27 (time=3560000)
MISMATCH: a=7 b=6 expected=42 got=18 (time=3590000)
MISMATCH: a=7 b=7 expected=49 got=21 (time=3620000)
MISMATCH: a=7 b=10 expected=70 got=54 (time=3710000)
MISMATCH: a=7 b=11 expected=77 got=49 (time=3740000)
MISMATCH: a=7 b=12 expected=84 got=36 (time=3770000)
MISMATCH: a=7 b=13 expected=91 got=35 (time=3800000)
MISMATCH: a=7 b=14 expected=98 got=42 (time=3830000)
MISMATCH: a=7 b=15 expected=105 got=45 (time=3860000)
MISMATCH: a=9 b=9 expected=81 got=65 (time=4640000)
```

```
MISMATCH: a=9 b=11 expected=99 got=83 (time=4700000)
MISMATCH: a=9 b=13 expected=117 got=101 (time=4760000)
MISMATCH: a=9 b=15 expected=135 got=247 (time=4820000)
MISMATCH: a=10 b=5 expected=50 got=34 (time=5000000)
MISMATCH: a=10 b=7 expected=70 got=118 (time=5060000)
MISMATCH: a=10 b=10 expected=100 got=68 (time=5150000)
MISMATCH: a=10 b=11 expected=110 got=78 (time=5180000)
MISMATCH: a=10 b=13 expected=130 got=114 (time=5240000)
MISMATCH: a=10 b=14 expected=140 got=236 (time=5270000)
MISMATCH: a=10 b=15 expected=150 got=166 (time=5300000)
MISMATCH: a=11 b=3 expected=33 got=61 (time=5420000)
MISMATCH: a=11 b=5 expected=55 got=39 (time=5480000)
MISMATCH: a=11 b=6 expected=66 got=122 (time=5510000)
MISMATCH: a=11 b=7 expected=77 got=81 (time=5540000)
MISMATCH: a=11 b=9 expected=99 got=83 (time=5600000)
MISMATCH: a=11 b=10 expected=110 got=78 (time=5630000)
MISMATCH: a=11 b=11 expected=121 got=229 (time=5660000)
MISMATCH: a=11 b=12 expected=132 got=244 (time=5690000)
MISMATCH: a=11 b=13 expected=143 got=127 (time=5720000)
MISMATCH: a=11 b=14 expected=154 got=162 (time=5750000)
MISMATCH: a=11 b=15 expected=165 got=137 (time=5780000)
MISMATCH: a=12 b=3 expected=36 got=52 (time=5900000)
MISMATCH: a=12 b=6 expected=72 got=104 (time=5990000)
MISMATCH: a=12 b=7 expected=84 got=68 (time=6020000)
MISMATCH: a=12 b=11 expected=132 got=212 (time=6140000)
MISMATCH: a=12 b=12 expected=144 got=208 (time=6170000)
MISMATCH: a=12 b=13 expected=156 got=220 (time=6200000)
MISMATCH: a=12 b=14 expected=168 got=136 (time=6230000)
MISMATCH: a=12 b=15 expected=180 got=164 (time=6260000)
MISMATCH: a=13 b=3 expected=39 got=55 (time=6380000)
MISMATCH: a=13 b=5 expected=65 got=121 (time=6440000)
MISMATCH: a=13 b=6 expected=78 got=110 (time=6470000)
MISMATCH: a=13 b=7 expected=91 got=67 (time=6500000)
MISMATCH: a=13 b=9 expected=117 got=101 (time=6560000)
MISMATCH: a=13 b=10 expected=130 got=242 (time=6590000)
MISMATCH: a=13 b=11 expected=143 got=223 (time=6620000)
MISMATCH: a=13 b=12 expected=156 got=220 (time=6650000)
MISMATCH: a=13 b=13 expected=169 got=145 (time=6680000)
MISMATCH: a=13 b=14 expected=182 got=134 (time=6710000)
MISMATCH: a=13 b=15 expected=195 got=171 (time=6740000)
MISMATCH: a=14 b=3 expected=42 got=50 (time=6860000)
MISMATCH: a=14 b=5 expected=70 got=118 (time=6920000)
MISMATCH: a=14 b=6 expected=84 got=100 (time=6950000)
MISMATCH: a=14 b=7 expected=98 got=74 (time=6980000)
MISMATCH: a=14 b=10 expected=140 got=236 (time=7070000)
MISMATCH: a=14 b=11 expected=154 got=194 (time=7100000)
MISMATCH: a=14 b=12 expected=168 got=200 (time=7130000)
MISMATCH: a=14 b=13 expected=182 got=134 (time=7160000)
MISMATCH: a=14 b=14 expected=196 got=148 (time=7190000)
MISMATCH: a=14 b=15 expected=210 got=186 (time=7220000)
MISMATCH: a=15 b=3 expected=45 got=49 (time=7340000)
MISMATCH: a=15 b=5 expected=75 got=115 (time=7400000)
MISMATCH: a=15 b=6 expected=90 got=98 (time=7430000)
MISMATCH: a=15 b=7 expected=105 got=77 (time=7460000)
MISMATCH: a=15 b=9 expected=135 got=247 (time=7520000)
MISMATCH: a=15 b=10 expected=150 got=230 (time=7550000)
MISMATCH: a=15 b=11 expected=165 got=201 (time=7580000)
MISMATCH: a=15 b=12 expected=180 got=196 (time=7610000)
MISMATCH: a=15 b=13 expected=195 got=139 (time=7640000)
MISMATCH: a=15 b=14 expected=210 got=154 (time=7670000)
MISMATCH: a=15 b=15 expected=225 got=181 (time=7700000)
FAIL: 256 vectors checked, 95 errors
```

1.3 GTKWave Waveform

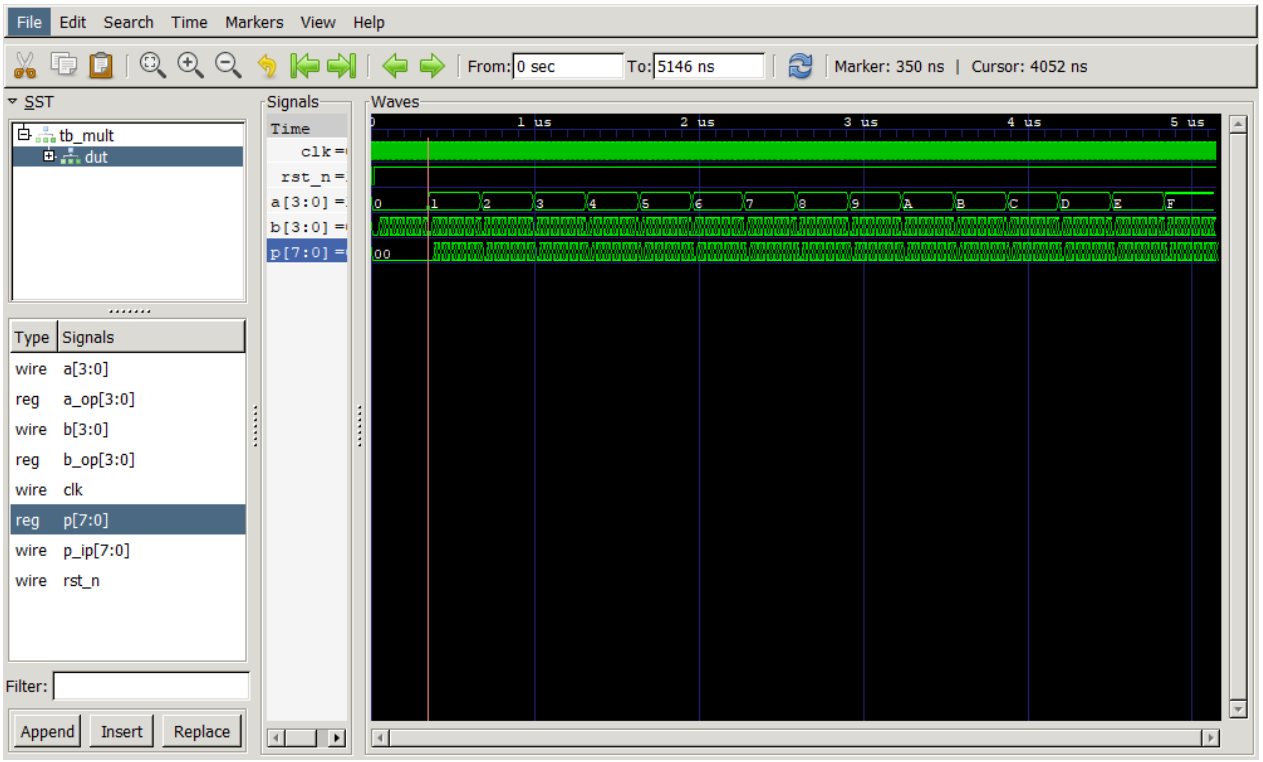


Figure 1: GTKWave view of the full RTL simulation.

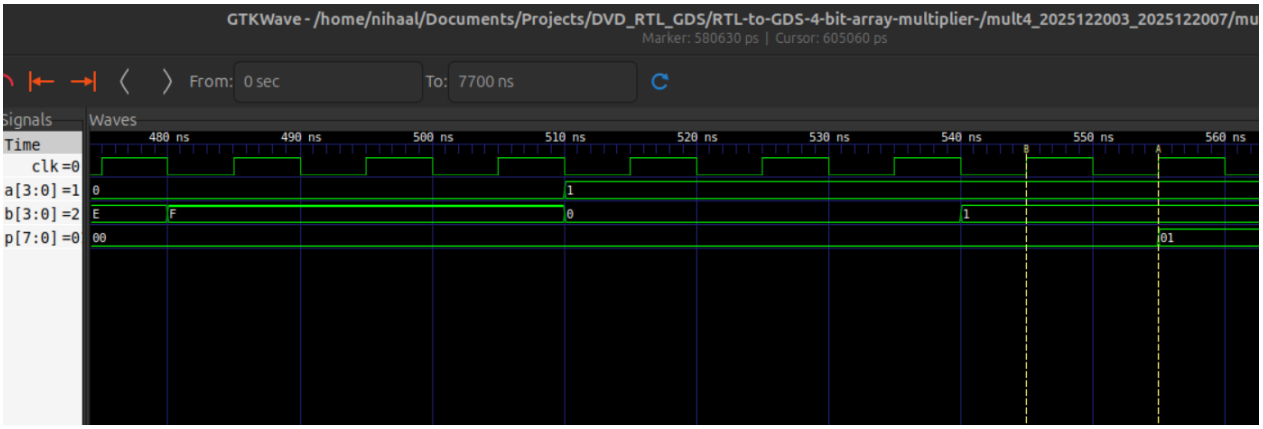


Figure 2: GTKWave view with markers.
It is clearly visible that it takes 2 clk edges for p to show the output of 01 (B marker being the starting point and A marker being the ending marker)

1.4 Hand-worked Example

The rule is $A = (\text{last digit of roll number}) + 5$ and $B = 7$.

Roll number	Last digit	A	B	$A \times B$
2025122003	3	8	7	56
2025122007	7	12	7	84

2025122003: 8×7

```
        1 0 0 0      A = 8
      x 0 1 1 1      B = 7
---------------
        1 0 0 0      pp0 = A AND b0 (b0 = 1)
      1 0 0 0        pp1 = A AND b1 (b1 = 1),
   shifted 1
    1 0 0 0          pp2 = A AND b2 (b2 = 1),
   shifted 2
  0 0 0 0            pp3 = A AND b3 (b3 = 0),
   shifted 3
---------------
0 0 1 1 1 0 0 0      P = 56 = 0x38
```

2025122007: 12×7

```
        1 1 0 0      A = 12
      x 0 1 1 1      B = 7
---------------
        1 1 0 0      pp0 = A AND b0 (b0 = 1)
      1 1 0 0        pp1 = A AND b1 (b1 = 1),
   shifted 1
    1 1 0 0          pp2 = A AND b2 (b2 = 1),
   shifted 2
  0 0 0 0            pp3 = A AND b3 (b3 = 0),
   shifted 3
---------------
0 1 0 1 0 1 0 0      P = 84 = 0x54
```

1.5 Block Diagram

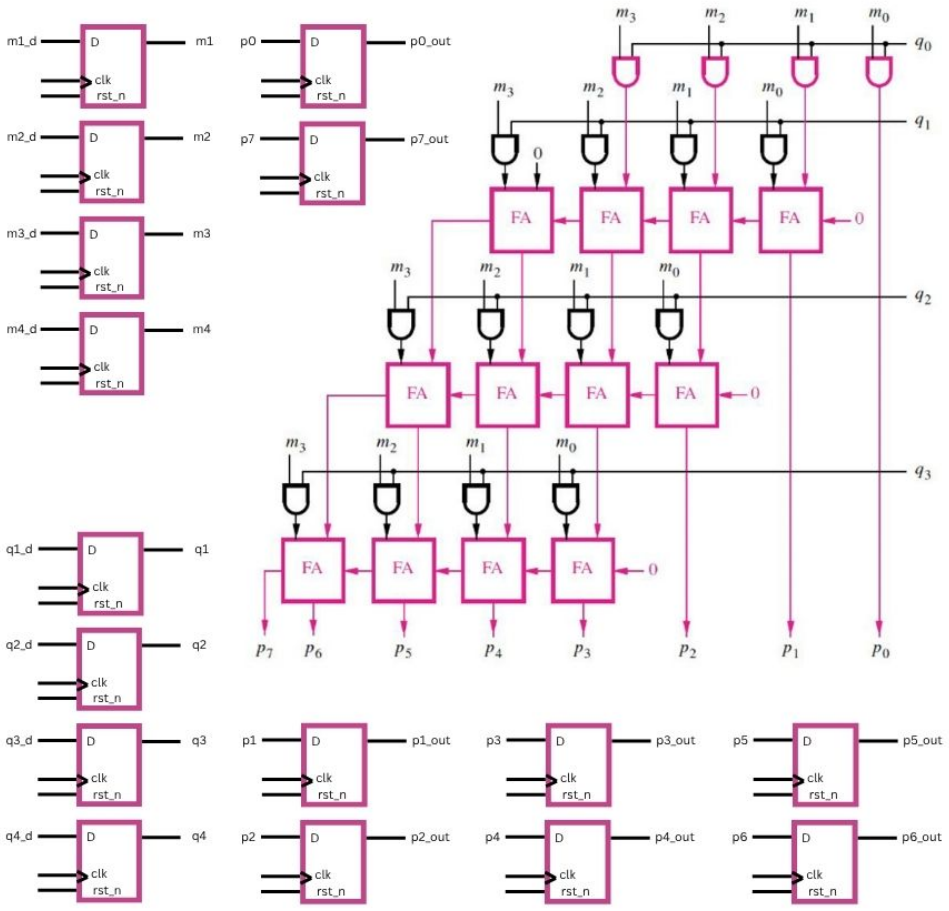


Figure 3: Block diagram of `mult_array`.

2 Part B

Table 1: Part B synthesis results.

Metric	Value
Total number of cells	63
Number of flip-flops	16
Total cell area (μm^2)	739.459
Flip-flop area (μm^2)	400.384
Flip-flop area (% of total)	54.15 %
The 3 most-used cell types and their counts	`sky130_fd_sc_hd__dfrtp_1` – 16
	`sky130_fd_sc_hd__nand2_1` – 12
	`sky130_fd_sc_hd__a21oi_1` – 6
Netlist simulation result (PASS/FAIL line)	`PASS: 256 vectors checked, 0 errors`

2.1 Netlist Simulation Pass Line

```
VCD info: dumpfile mult.vcd opened for output.
PASS: 256 vectors checked, 0 errors
```

3 Part C: Static Timing Analysis Results

Item	Value
Worst slack at $T = 10$ ns	7.494 ns
T_{min}	2.51 ns (2.506 before rounding up)
F_{max}	about 398 MHz
Slack at T_{min}	0.004 ns
Critical path start point	`_098_` (`dfrtp_1` flop)
Critical path end point	`_092_` (`dfrtp_1` flop)
Number of cells on critical path	8 combinational cells (excluding launch and capture flops)
Latency ($2 \times T_{min}$)	5.02 ns
Throughput	about 398 million results/s

List of cells on critical path

Delay (ns)	Arrival (ns)	Edge	Instance (cell)
0.180	0.568	v	`_039_/X` (`sky130_fd_sc_hd__and2_0`)
0.201	0.769	^	`_044_/Y` (`sky130_fd_sc_hd__o21ai_0`)
0.131	0.900	v	`_046_/Y` (`sky130_fd_sc_hd__nand3_1`)
0.384	1.284	v	`_070_/X` (`sky130_fd_sc_hd__maj3_1`)
0.155	1.439	v	`_071_/Y` (`sky130_fd_sc_hd__xnor2_1`)
0.379	1.818	v	`_079_/X` (`sky130_fd_sc_hd__maj3_1`)
0.379	2.197	v	`_083_/X` (`sky130_fd_sc_hd__maj3_1`)
0.175	2.372	v	`_085_/X` (`sky130_fd_sc_hd__xnor3_1`)

4 Part D

The flow needed 5 runs to close timing; the final run used a clock period of 6 ns.

4.1 Results (final run, 6 ns)

$$T_{min}(\text{post-route}) = 6.0 - 0.0115 = 5.9885 \text{ ns}, \qquad F_{max}(\text{post-route}) = \frac{1000}{5.9885} \approx 167 \text{ MHz}$$

Metric	Value
CLOCK_PERIOD used (ns)	6
Number of runs needed	5
Core area (μm^2)	17759.5
Cells after synthesis (from Part B)	63
Cells after place and route	93 (338 with tap cells, 1899 with tap and fill cells)
Kinds of cells added	Timing repair buffers (23), clock buffers (3), tap cells (245), fill cells (1561)
Wirelength (μm)	2387
Setup slack (ns)	+0.0115 (worst corner, `max_ss_100C_1v60`)
Post-route F_{max} (MHz)	about 167
F_{max} from Part C (MHz)	about 398
DRC errors	0
LVS errors	0

4.2 Final Layout (GDSII)

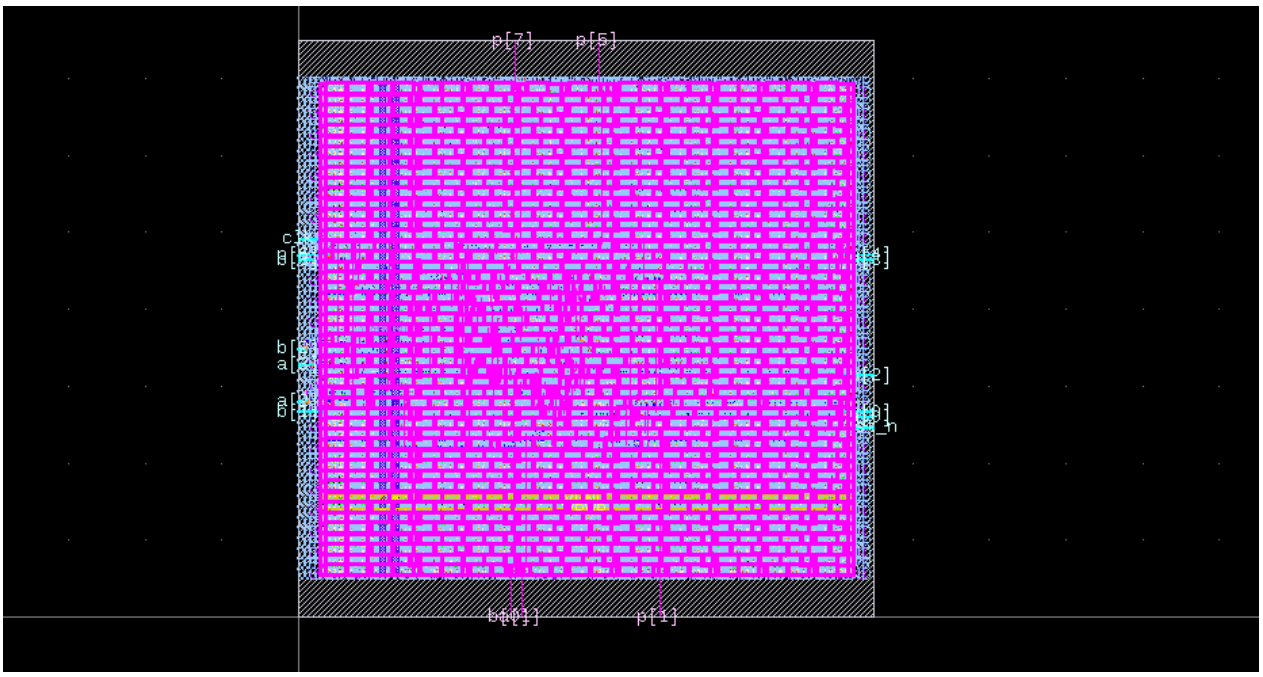


Figure 4: Complete GDSII layout of the 4-bit array multiplier.

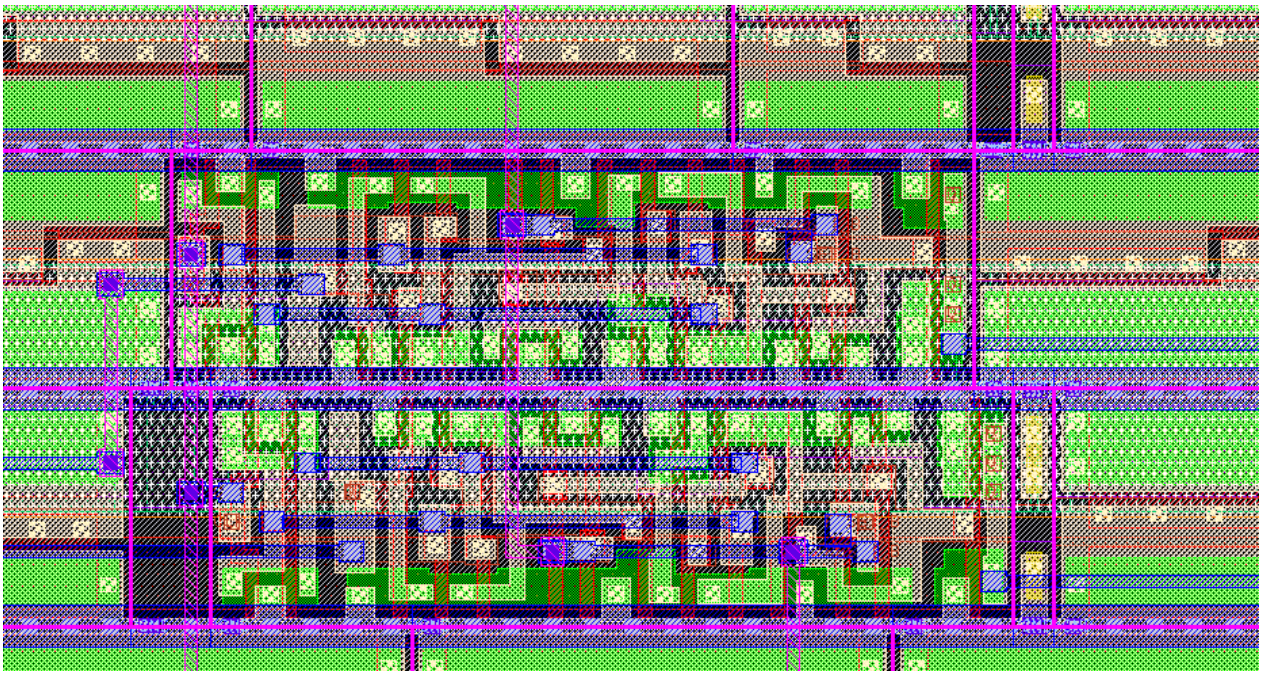


Figure 5: Zoomed-in view of the GDSII layout.

5 Questions

Q1. Logic cell count. Synthesis produced 63 cells, and 16 of them are flip-flops, so the logic is $63 - 16 = 47$ cells. This is not $16 + 12 = 28$ because the tool maps the logic onto library gates, not one cell per AND gate or full adder. A full adder needs more than one library cell (maybe, one for the sum and one for the carry). The synthesis tool also optimizes across the whole design, so some logic is merged into gates that were not originally intended in the RTL, such as `nand` and `a21oi` cells, which changes the count.

Q2. Critical path. The critical path runs from flop `_098_` to flop `_092_`. There are 8 combinational cells between the two flops (`and2`, `o21ai`, `nand3`, `maj3`, `xnor2`, `maj3`, `maj3`, `xnor3`). The estimate is about $2N = 8$ full-adder delays. The path also has 8 cells, but these cells are not full adders one to one, because the synthesis tool mapped the logic onto library cells. So the cell count is close to the estimate, though the two numbers count different things.

Q3. Flip-flop area. The flip-flops take $400.384\,\mu\text{m}^2$ of the $739.459\,\mu\text{m}^2$ total cell area, which is $400.384/739.459 \approx 54.15\%$. The percentage is large because each flip-flop is much bigger than a logic gate: one `dfrtp_1` is $400.384/16 = 25.02\,\mu\text{m}^2$, while one `nand2_1` is $45.043/12 = 3.75\,\mu\text{m}^2$, about 6.7 times smaller. The design has 16 flip-flops (8 input and 8 output registers) for only 47 small logic cells, so the 16 flops make up about a quarter of the cells but over half of the area.

Q4. Post-route F_{max}. Post-route $F_{max} = 1000/(6 - 0.0115) \approx 167\,\text{MHz}$, against about 398 MHz in Part C. Part C analysed the synthesized netlist at one corner (typical, 1.80 V, 25 °C) with an ideal clock. The place-and-route flow checks several corners, and its worst setup slack is in the slowest one (1.60 V, 100 °C), where gates are slower. At the typical corner the same flow has a setup slack of 2.935 ns, which gives $1000/(6 - 2.935) \approx 326\,\text{MHz}$, so most of the drop comes from the corner. The rest comes from real wire delay, a real clock tree, and a different netlist produced by the flow's own synthesis and optimisation.

The flow added four kinds of cells:

- **Timing repair buffers (23):** fix timing and signal slew on long or heavily loaded wires.

- **Clock buffers (3):** form the clock tree that delivers `clk` to all 16 flip-flops.
- **Tap cells (245):** connect the substrate and wells to the power rails, which prevents floating wells and hence latch-up.
- **Fill cells (1561):** fill the empty gaps in the cell rows so that the layout density is continuous.

Q5. DRC and LVS. A DRC check verifies that the layout shapes follow the foundry's manufacturing rules, such as minimum width and spacing. An LVS check verifies that the circuit extracted from the layout is the same as the netlist the design started from. Both must pass because a layout can obey every rule and still be the wrong circuit (LVS fails), or be the right circuit and still be impossible to manufacture (DRC fails). Our final run has 0 DRC errors and 0 LVS errors.

Q6. Scaling to 8 bits. Using the formulas from Section 2, for an N-bit array multiplier:

- Full adders: $N(N-1)$. For $N = 4$ this is $4 \times 3 = 12$. For $N = 8$ it is $8 \times 7 = 56$, which is about $56/12 \approx 4.7$ times as many.
- Critical path: about $2N$ full-adder delays. For $N = 4$ this is $2 \times 4 = 8$. For $N = 8$ it is $2 \times 8 = 16$, so the delay doubles.

The number of full adders grows as a square (about 4.7 times for a doubled width), while the critical path delay grows linearly (2 times for a doubled width).

6 Contributions

Part	Contribution
Part A	Nihaal: RTL design of the multiplier (`mult_array`). Devang: RTL design of the full adder, final testbench and testing.
Part B	Devang
Part C	Nihaal
Part D	Nihaal and Devang
Questions	Answered together by Nihaal and Devang